AI ASSISTED CODING

NAME : R . ASHMITHA
BATCH : 40
ROLL NO : 2303A52111

Lab 15 – Backend API Development: Creating RESTful services with AI

Task Descriptor-1: (Setup Fast API Backend)

Instructions:

• Install Fas tAPI and Uvicorn.

• Use AI to generate a basic FastAPI application with one endpoint

Sample Code:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def read_root():

return {"message": "Welcome to AI-powered FastAPI"}
```

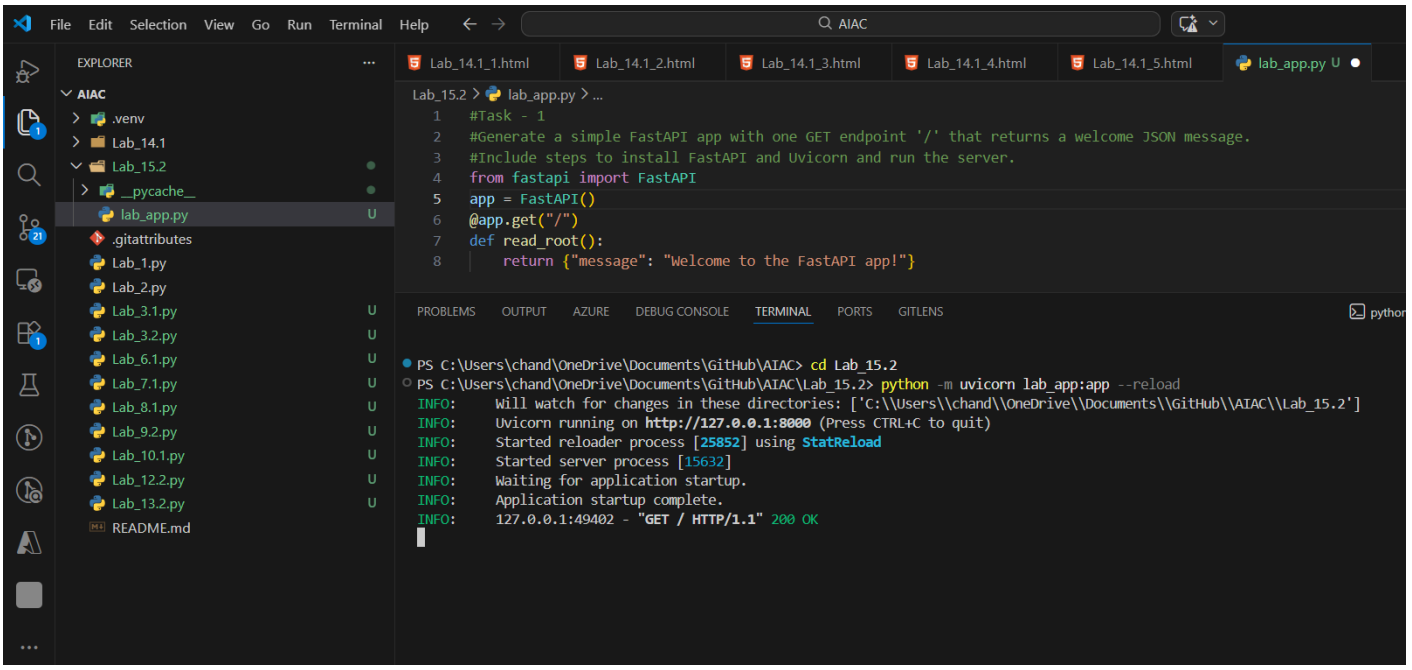
Expected Output:

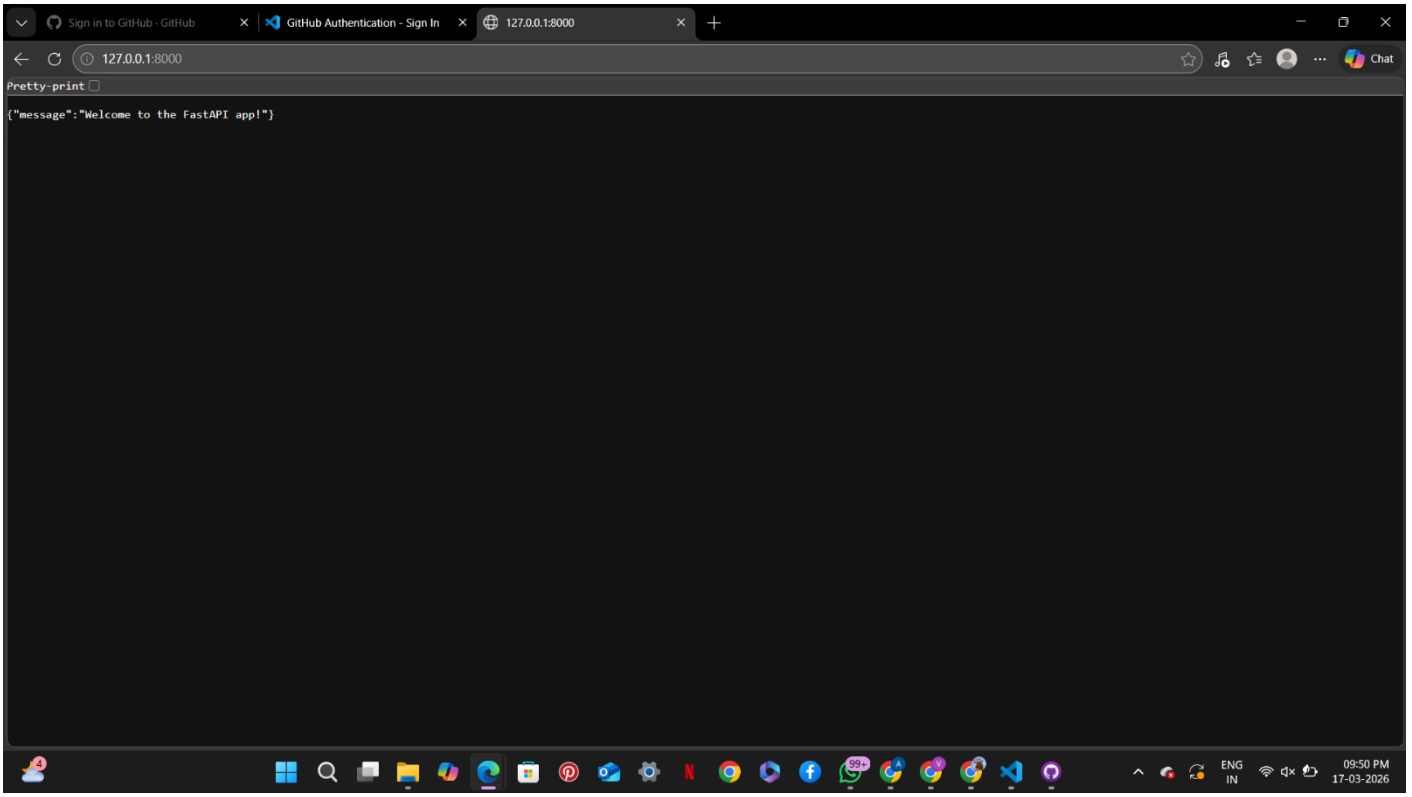
Running the server and accessing:

http://127.0.0.1:8000/

Response:

{"message": "Welcome to AI-powered FastAPI"}





ANALYSIS

- Installs FastAPI and Uvicorn for backend development.
- Creates a basic API with one GET endpoint.
- Returns a JSON response to the client.
- Demonstrates how to run a server locally.
- Builds foundation for further API tasks.

Task Descriptor- 2: (API – Create and Read Items)

Instructions:

• Implement endpoints to add and retrieve products.

• Use a Python list as temporary storage.

Sample Code:

```
items = []

@app.get("/items")

def get_items():

return items

@app.post("/items")

def add_item(item: dict):

items.append(item)

return {"message": "Item created", "item": item}
```

Expected Output:

GET /items → []

POST /items with {"name":"Pen","price":20}

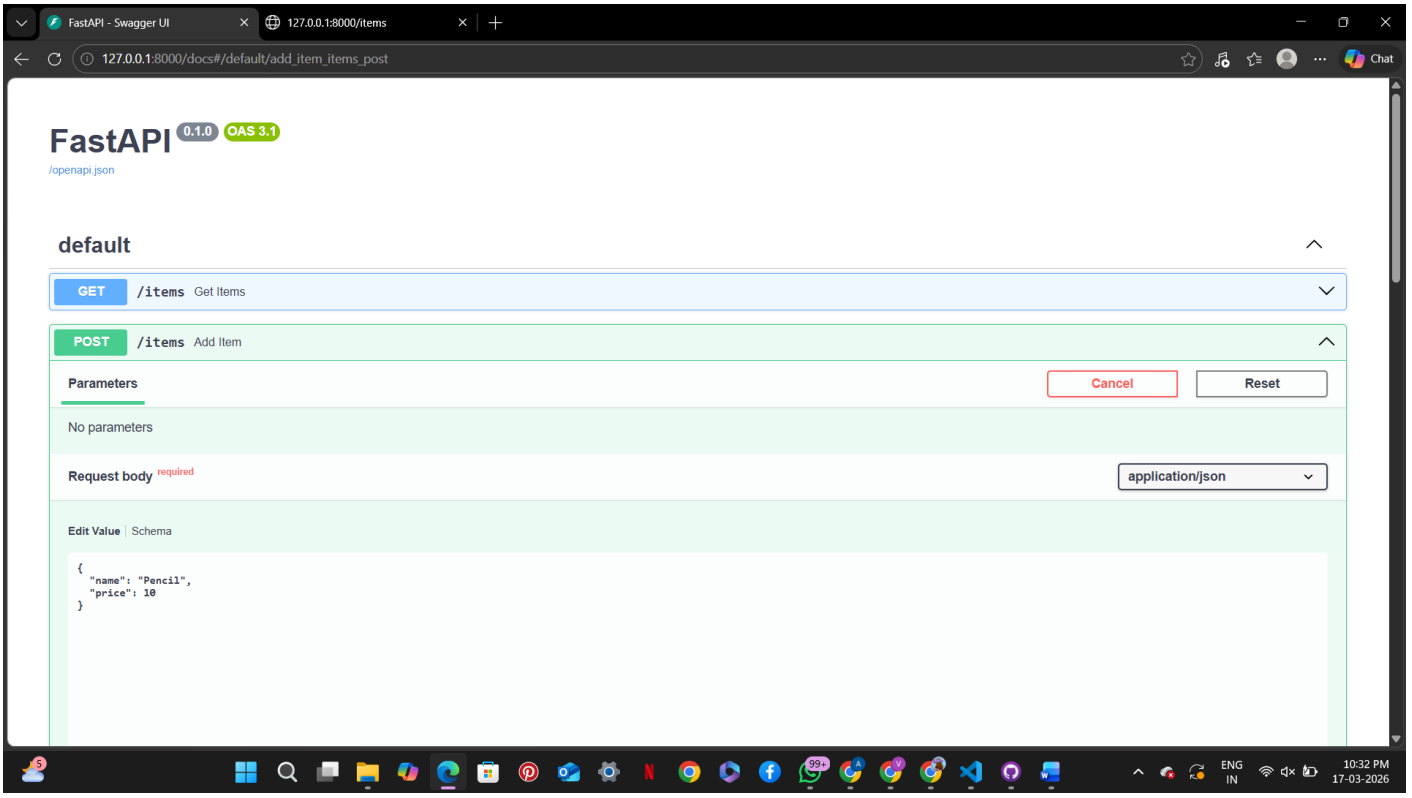
→ {"message":"Item created","item":{"name":"Pen","price":20}}

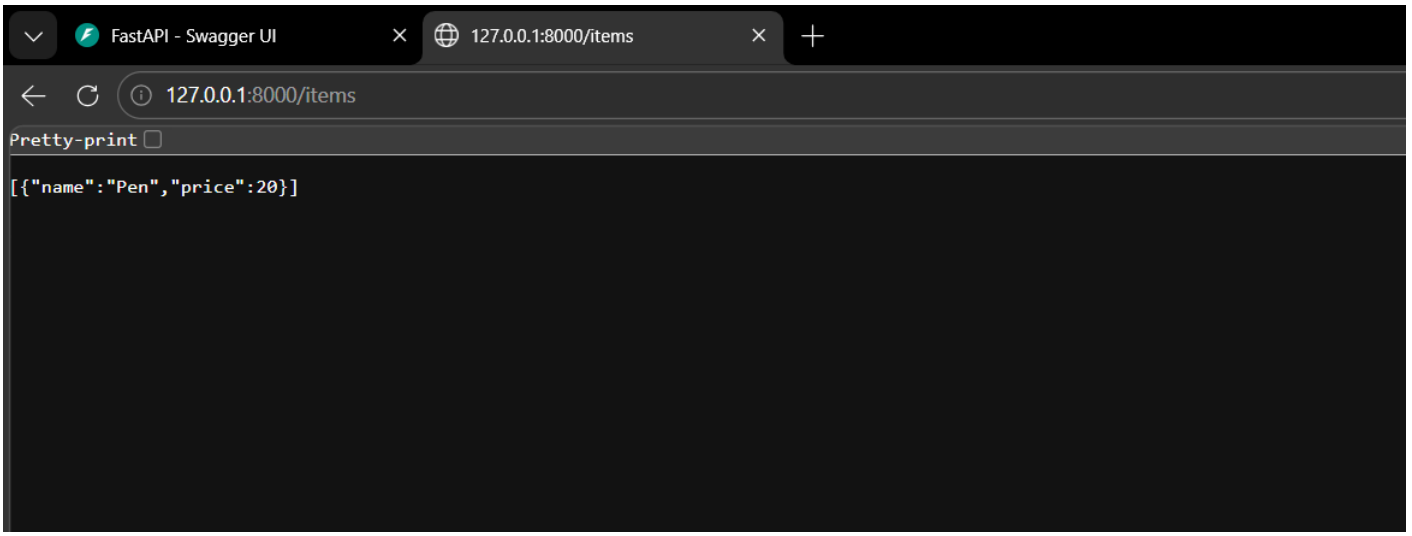
Lab_15.2 > lab_app.py > add_item

```python
#Task - 2
#Create FastAPI endpoints to GET and POST items using a Python list as storage. POST should add item, GET should return all items.
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

PROBLEMS OUTPUT AZURE DEBUG CONSOLE TERMINAL PORTS GITLENS python - Lab_15.2

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO:     Stopping reloader process [1708]
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\chand\\OneDrive\\Documents\\GitHub\\AIAC\\Lab_15.2']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [11760] using StatReload
INFO:     Started server process [5064]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:53743 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:60648 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:60648 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:56450 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:63568 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51910 - "GET /items HTTP/1.1" 200 OK
```





ANALYSIS

- Implements GET and POST methods.
- Uses a Python list as temporary storage.
- Allows adding new items dynamically.
- Retrieves all stored items.
- Introduces basic CRUD operations.

Task Descriptor- 3: (Update Existing Item)

Instructions:

• Use AI to generate an endpoint to update an item using its index.

Sample Code:

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
if index >= len(items):
return {"error": "Item not found"}
items[index] = item
return {"message": "Item updated", "item": item}
```

Expected Output:

PUT /items/0 with {"name":"Pencil","price":10}

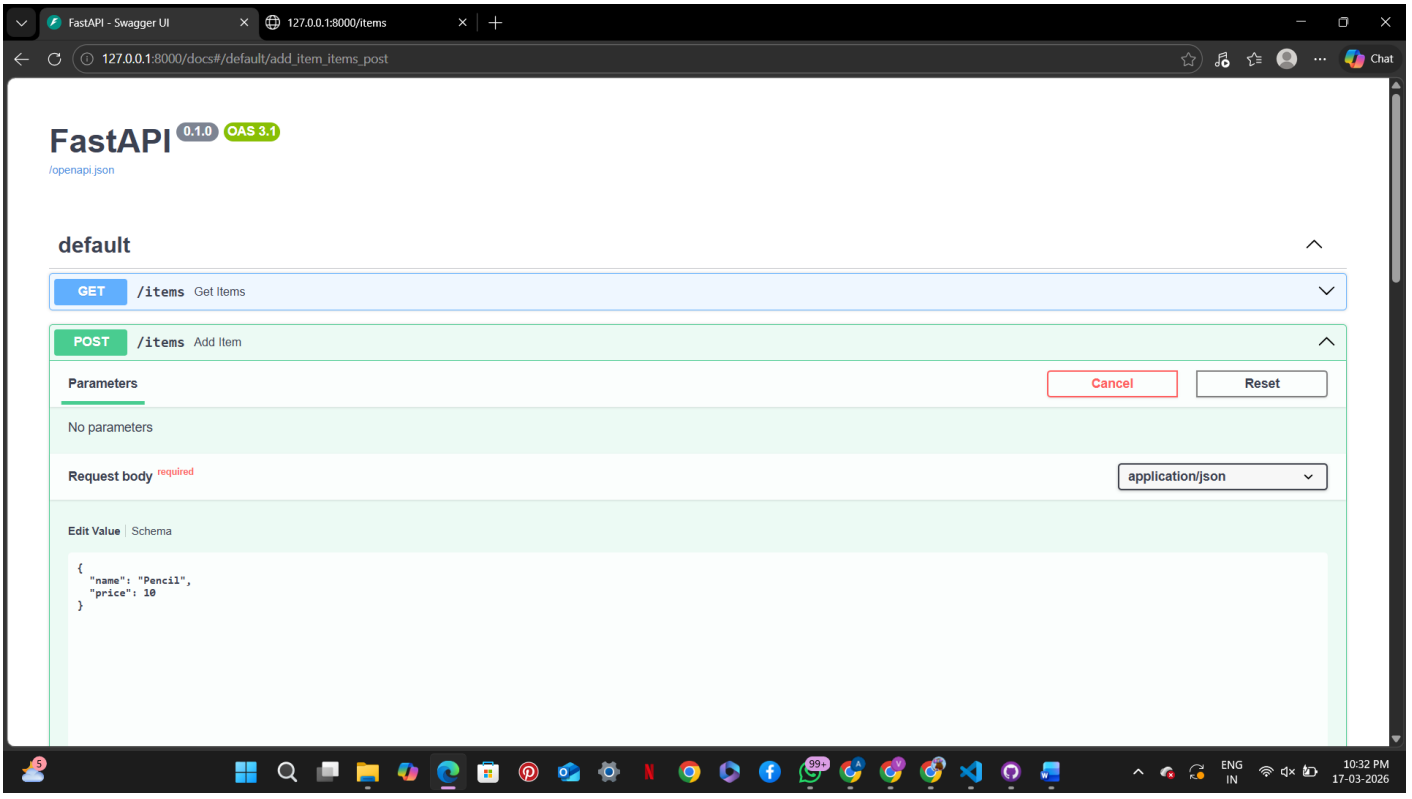
→ {"message":"Item updated","item":{"name":"Pencil","price":10}}

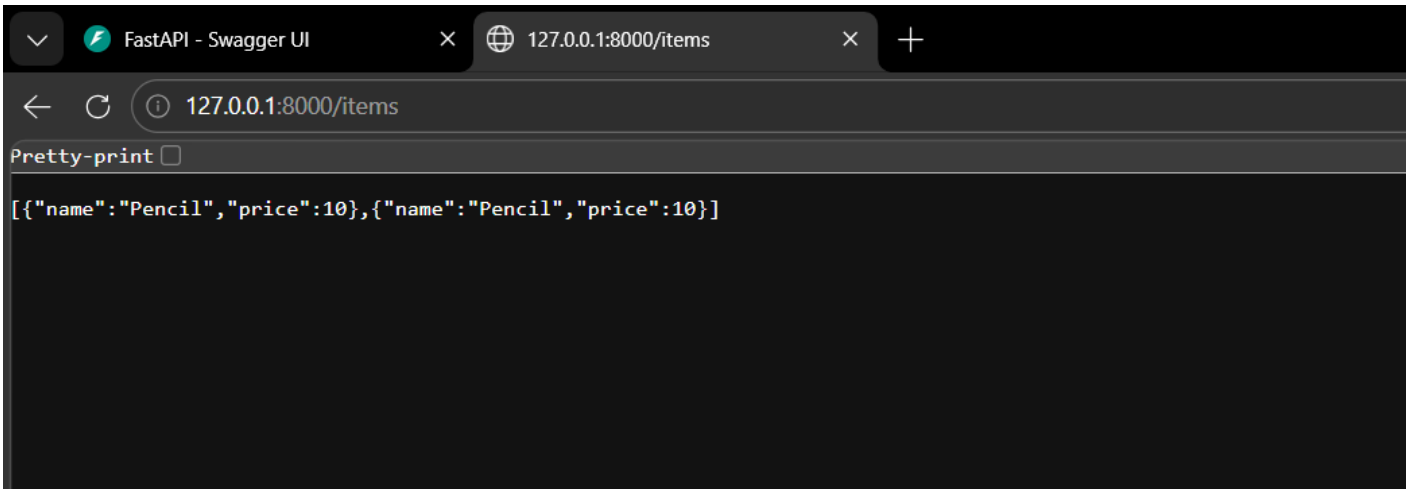
Lab_15.2 > lab_app.py > ...

```python
#Task - 3
#Create a FastAPI PUT endpoint to update an item by index in a list. Handle case when item is not found.
from fastapi import FastAPI, HTTPException
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
```

PROBLEMS OUTPUT AZURE DEBUG CONSOLE TERMINAL PORTS GITLENS pyth

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\chand\\OneDrive\\Documents\\GitHub\\AIAC\\Lab_15.2']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [24780] using StatReload
INFO:     Started server process [24312]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:57721 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:58416 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:58416 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:51492 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:51492 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:64316 - "GET /items HTTP/1.1" 200 OK
```





ANALYSIS

- Uses PUT method to update existing data.
- Updates item based on index position.
- Checks for invalid index (error handling).
- Replaces old data with new data.
- Improves data manipulation skills.

Task Descriptor- 4: (Delete Item)

Instructions:

• Implement deletion of an item by index.

Sample Code:

```
@app.delete("/items/{index}")
def delete_item(index: int):
if index >= len(items):
return {"error": "Item not found"}
removed = items.pop(index)
return {"message": "Item removed", "item": removed}
```

Expected Output:

DELETE /items/0

→ {"message":"Item removed","item":{"name":"Pencil","price":10}}

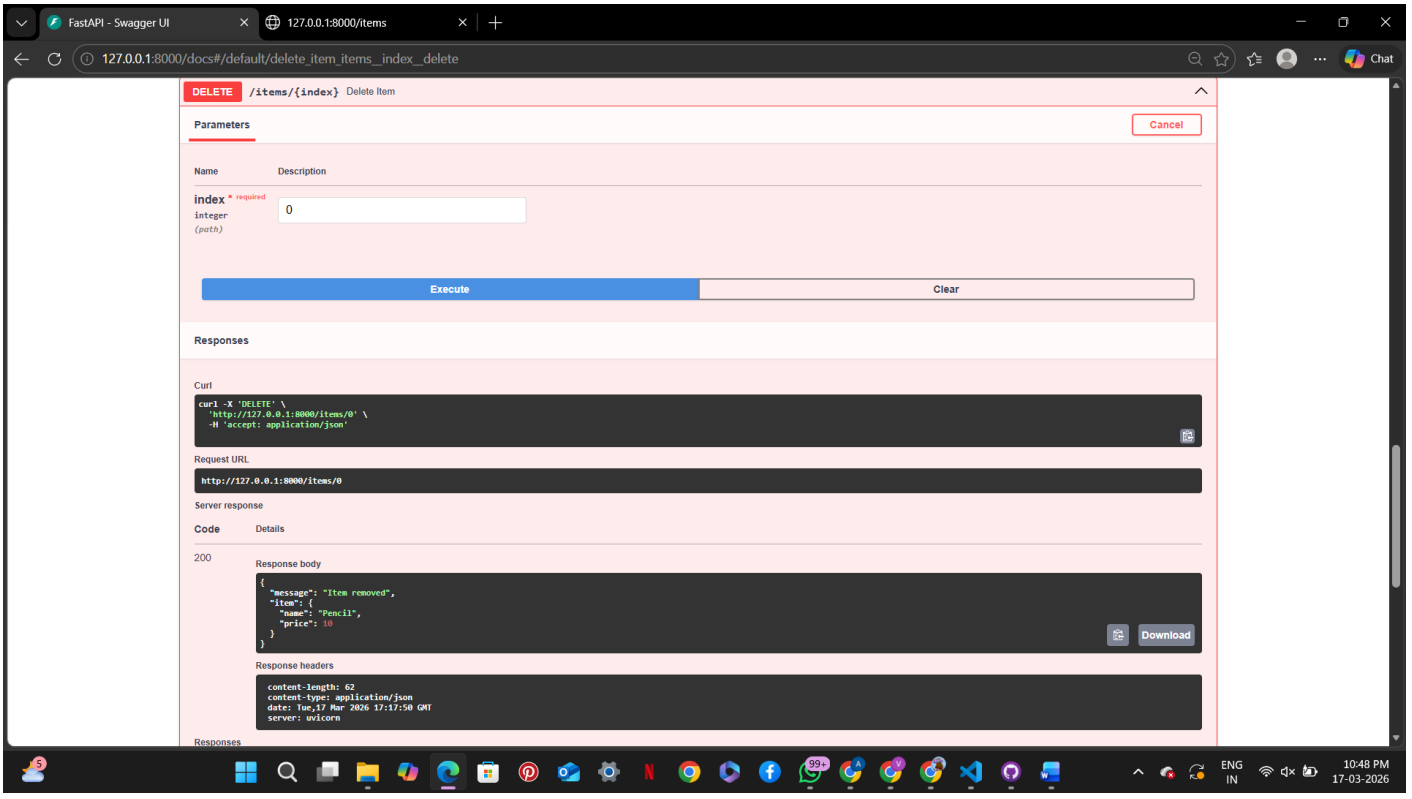
Lab_15.2 > lab_app.py > ...

```python
#Task - 4
#Create a FastAPI DELETE endpoint to remove an item by index from a list. Return error if index is invalid.
from fastapi import FastAPI, HTTPException
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    if index < 0 or index >= len(items):
        return {"error": "Invalid index"}

    removed_item = items.pop(index)
    return {"message": "Item removed", "item": removed_item}
```

PROBLEMS OUTPUT AZURE DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO:     127.0.0.1:57163 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:57163 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:61168 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:63071 - "DELETE /items/0 HTTP/1.1" 200 OK
INFO:     127.0.0.1:63071 - "DELETE /items/0 HTTP/1.1" 200 OK
INFO:     127.0.0.1:53414 - "GET /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:56249 - "POST /items HTTP/1.1" 200 OK
INFO:     127.0.0.1:57746 - "DELETE /items/0 HTTP/1.1" 200 OK
```



ANALYSIS

- Uses DELETE method to remove items.

- Deletes item using index value.
- Validates index before deletion.
- Returns removed item as response.
- Ensures safe data handling.

Task Descriptor -5: (Add Auto-Generated Documentation)

Instructions:

• Use AI to add inline comments and docstrings for all endpoints.

• Optionally, integrate Swagger / Flask-RESTX to auto-generate

API documentation.

Sample Code:

```
@app.route('/items', methods=['GET'])
def get_items():
"""
GET /items
Returns a list of all items in the store.
"""
return jsonify(items)
```

Expected Output:

• Clear documentation for all endpoints, e.g., via /docs if using

Swagger.

• Students should be able to see endpoint description, methods, and sample payloads.

Lab_15.2 > lab_app.py > get_items

```python
#Task - 5
#Add docstrings and comments to all FastAPI endpoints and enable auto-generated Swagger docs (/docs) with clear descriptions and s
from fastapi import FastAPI, HTTPException
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Invalid index")

    removed_item = items.pop(index)
    return {"message": "Item removed", "item": removed_item}
```

PROBLEMS OUTPUT AZURE DEBUG CONSOLE TERMINAL PORTS GITLENS powershell - Lab_15.2

```
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [9248]
INFO:     Stopping reloader process [8]
PS C:\Users\chand\OneDrive\Documents\GitHub\AIAC\Lab_15.2>
```

FastAPI - Swagger UI

127.0.0.1:8000/docs

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

GET /items Get Items

POST /items Add Item

PUT /items/{index} Update Item

DELETE /items/{index} Delete Item

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

ANALYSIS

- Adds docstrings to explain endpoints.
- Enables automatic Swagger documentation.
- Helps users understand API usage easily.
- Displays request/response formats.
- Improves readability and usability of API.